

**Chuletarlo
char lward;**

Index

- ❖ Quick Stuff (3)
- ❖ Data Structures (4)
- ❖ Graph Algorithms (7)
- ❖ Dynamic Programming (13)
- ❖ Number Theory (14)
- ❖ Geometry (16)
- ❖ Ada Language (20)

QUICK STUFF

String to lower

`transform(s.begin(), s.end(), s.begin(), ::tolower);`

Custom Sort

```
bool cmp( tipo a, tipo b){
    return a > b;
}
sort(A.begin(), A.end(), cmp);
----- //2a forma con struct
struct cmp{
    bool operator()(tipo a, tipo b){
        return a > b;
    }
};
sort(A.begin(), A.end(), cmp());
priority_queue<tipo, vector<tipo>, cmp> pq;
```

String to Int

```
int stringToInt(string s){
    int temp = 0;
    for(int i = 0; i < s.length(); i++){
        temp = temp*10+(s[i] - '0');
    }
    return temp;
}
```

Bitmask (<= 62 values)

& and

| or

^ xor

~ not

Check bit status

`T = num & (1<<index)`

Turn off a bit

`num &= ~(1<<index)`

Flip a bit

`num ^= (1<<index)`

Turn on all bits

`S = (1<<n)-1`

`#define LSONE(S) ((S) & -(S))` : least significant one

`__builtin_clz()` `__lzcnt()`: number of zeros on the left

`__builtin_ctz()` : number of zeros on the right

`__builtin_popcount()` `__popcnt`: number of ones

Getline

Después de hacer `getline(cin,s)`, hacer `cin.ignore()`

StringStream

```
stringstream ss (s);
while(ss >> word)
    //do something
```

DATA STRUCTURES

UFDS $O(\log n)$

```
struct UFDS{
    vector<ll> p, setSize;
    int numSets;
    UFDS(int n){
        p.assign(n, 0);
        setSize.assign(n, 1);
        numSets = n;
        for(int i = 0; i < n; i++) p[i] = i;
    }
    int find(int x){return p[x] == x ? x : p[x] = find(p[x]);}
    void merge(int x, int y){
        int i = find(x); int j = find(y);
        if(i==j) return;
        p[i] = j;
        setSize[j] += setSize[i];
        numSets--;
    }
    int size(int i) {return setSize[find(i)];}
};
```

Fenwick Tree $O(\log n)$

1-index (ie. $rsq(1, n)$ gives the whole sum, $rsq(0, 1)$ segmentation fault.)

PURQ for sum queries. $O(n \log n)$ for building and $O(\log n)$ for queries (both rsq and update)

```
struct FenwickTree{
    vector<int> ft;
    FenwickTree(int m){
        ft.assign(m+1, 0);
    }
    FenwickTree(const vector<int>& arr){
        ft.resize(arr.size()+1);
        for(int i = 1; i <= arr.size(); i++)
            update(i, arr[i]);
    }
    int rsq(int j){
        int sum = 0;
        for(; j; j -= LSOne(j))
            sum += ft[ j ];
        return sum;
    }
    int rsq(int i, int j){
        return rsq( j ) - rsq( i - 1 );
    }
    void update(int i, int v){
        for(; i < ft.size(); i += LSOne(i))
            ft[ i ] += v;
    }
};
```

Trie O(n)

prefixes : numero de palabras que empiezan con ese prefijo

words : numero de palabras que son exactamente la del nodo

time complexity :

building trie: $O(n+l)$ n := num of words, l := avrg length per word. queries

query: $O(n)$ n := length of searched word

```
struct Trie{
    int prefixes, words;
    vector<Trie*> child;
    Trie() : prefixes(0), words(0), child(26, nullptr) { }
    ~Trie() {for(int i = 0; i < 26; i++) delete child[i]; }
    void add(string const& s, int i = 0){
        ++prefixes;
        if(i >= s.size()) words++;
        else (child[s[i]-'a'] == nullptr ? child[s[i] - 'a'] = new Trie() :
child[s[i] - 'a']->add(s,i+1);
    }
};
```

Sparse Table O(1)

0-index

min/max range queries. $O(n \log n)$ pre computation and $O(1)$ per query

```
struct sparseTable{
    vector<vector<int>> st;
    int n,m;
    int lg2(int i){
        return 31 - __builtin_clz(i);
    }
    sparseTable(const vector<int>& v) : n(v.size()), m(lg2(v.size())){
        st.assign(m+1, vector<int>(n));
        for(int j = 0; j < n; j++) st[0][j] = v[j];
        for(int i = 1; i <= m; i++)
            for(int j = 0; j + (1 << i) <= n; j++)
                st[i][j] = min(st[i-1][j], st[i-1][j + (1 << (i-1))]);
    }
    int query(int L, int R){
        int i = lg2(R-L+1);
        return min(st[i][L], st[i][R + 1 - (1 << i)]);
    }
};
```

Segment Tree O(logn)

0-index

RURQ

```
struct SegmentTree {
    //Para cambiar RANGE QUERIES entre incremento o sustitucion, cambiar donde ponga 'CAMBIO'
    //Para cambiar el tipo de operacion, cambiar donde ponga 'OPERACION'
    vector<ll> st, lazy;
    ll n, neutral = LLINF; //OPERACION
    ll defecto = 0; // CAMBIO

    ll op(ll a, ll b){ //OPERACION
        return min(a,b);
    }
};
```

```

int l(int v) {return v << 1;}
int r (int v) {return ((v << 1) + 1);}

SegmentTree(vi A) {
    n = (ll)A.size();
    while (__builtin_popcount(n) != 1) {
        A.push_back(neutral);
        n++;
    }
    st.resize(n << 1);
    lazy.resize(n << 1, defecto);
    for (ll i = 0; i < n; i++)
        st[n + i] = A[i];
    for (ll i = n - 1; i >= 1; i--)
        st[i] = op( st[l(i)], st[r(i)] );
}

void propagate(int node, int L, int R){
    if(lazy[node] != defecto){
        st[node] += lazy[node]; //CAMBIO
        if(L != R){
            lazy[l(node)] += lazy[node]; //CAMBIO
            lazy[r(node)] += lazy[node]; //CAMBIO
        }
        lazy[node] = defecto;
    }
}

ll ROpQ(int node, int L, int R, int ql, int qr) { //current node, maxLeft, maxRight,
querieLeft, querieRight
    propagate(node, L, R);
    if (ql <= L && R <= qr)
        return st[node];
    if (L > qr || ql > R)
        return neutral;
    ll mid = (L + R) / 2;
    return op(ROpQ(l(node), L, mid, ql, qr) , ROpQ(r(node), mid+1, R, ql, qr));
}

ll ROpQ(int ql, int qr) { //querieLeft, querieRight
    return ROpQ(1, 0, n - 1, ql, qr);
}

void update(int node, int L, int R, int ql, int qr, ll val) { //current node, maxLeft,
maxRight, querieLeft, querieRight
    propagate(node, L, R);
    if(ql <= L && R <= qr){
        lazy[node] += val; //CAMBIO
        propagate(node, L, R);
    }else if(L > qr || ql > R){
        return;
    }else{
        int mid = (L + R) / 2;
        update(l(node), L, mid, ql, qr, val);
        update(r(node), mid+1, R, ql, qr, val);
        st[node] = op(st[l(node)], st[r(node)]);
    }
}

void update(int ql, int qr, ll val){ update(1, 0, n-1, ql, qr, val); } //querieLeft,
querieRight
void show(){
    for(int i = 0; i < n; i++) cout << st[n+i] << " ";
    cout << " ";
}
};

```

GRAPH ALGORITHMS

Dijkstra $O(E \cdot \log V)$

adj : [from : [{to, dist}, ...]] (primer elem del pair es el vértice destino y el segundo el cost)

```
const int INF = INT_MAX;
void dijkstra(int s, vector<int> & d, vector<int> & p, vvii& adj) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    priority_queue<ii, vii, greater<ii> > q;
    q.push(make_pair(0,s));
    while (!q.empty()) {
        int v = q.top().second, dV = q.top().first; q.pop();
        if(dV != d[v]) continue;
        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push({d[to], to});
            }
        }
    }
}
```

Bellman Ford $O(V \cdot E)$

1-index

arr := vvi arr(n, vi(3)) -> {u, v, w}

d = {INF for all} d[source] = 0;

p = {-1 for all} parents

Hayar vertices con ciclos negativos y luego comprobar si mis caminos de s a t pasan por alguno de estos.

```
void bellmanFord(const vvi& arr, vector<ll>& d, vi& p){
    int n = d.size()-1, m = arr.size();
    for(int i = 0; i < n-1; i++){
        for(int j = 0; j < m; j++){
            int u = arr[j][0], v = arr[j][1], w = arr[j][2];
            if(d[u] < INF && d[v] > d[u] + w){
                d[v] = d[u] + w;
                p[v] = u;
            }
        }
    }
    //Checking for negative cycles
    for (int j = 0; j < m; j++) {
        int u = arr[j][0], v = arr[j][1], w = arr[j][2];
        if (d[u] < INF && d[u] + w < d[v]) {
            // negative cycle found
        }
    }
}
```

Floyd $O(V^3)$

d : matriz con distancias (d[i][j] dist de i a j)

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

Articulation Points $O(E + V)$

vvi adj : adjacency graph

run find_cutpoint(adj) and everytime a cutpoint is found, **IS_CUTPOINT()** will be called.

```
void IS_CUTPOINT(int u){}
void dfs(int v, const vvi& adj, vi& low, vi& tin, vector<bool> visited, int& timer, int p = -1)
{
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, adj, low, tin, visited, timer, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints(const vvi& adj) {
    int n = adj.size();
    int timer = 0;
    vector<bool> visited(n, false);
    vi tin(n, -1), low(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i, adj, low, tin, visited, timer);
    }
}
```

Lowest Common Ancestor $O(\log n)$

Implementation using binary lifting. $O(n \log n)$ for preprocessing and $O(\log n)$ for each query

```
struct LCA{
    int n, l;
    vvi up;
    vi d;
    LCA(const vvi& adj, int root = 0){
        n = adj.size();
        l = ceil(log2(n));
        up.assign(n, vi(l));
        d.assign(n, 0);
        dfs(root, adj);
    }
    void dfs(int u, const vvi& adj){
```



```

        for(int v : adj[u]){
            d[v] = d[u] + 1;
            up[v][0] = u;
            for(int i = 1; i < l; i++)
                up[v][i] = up[up[v][i-1]][i-1];
            dfs(v, adj);
        }
    }
    int lca(int u, int v){
        if(d[u] < d[v]) swap(u,v);
        int k = d[u]-d[v];
        for(int i = l - 1; i >= 0; i--){
            if(k & (1<<i))
                u = up[u][i];
            if(u == v) return u;
            for(int i = l-1; i >= 0; i--){
                if(up[u][i] != up[v][i]){
                    u = up[u][i];
                    v = up[v][i];
                }
            }
        }
        return up[u][0];
    }
};

```

Khans Algo $O(E+V)$

toposort

```

priority_queue<int, vi, greater<int>> pq;
for(int i = 0; i < n; i++)
    if(in_degree[i] == 0)
        pq.push(i);
while(!pq.empty()){
    int u = pq.top(); pq.pop();
    for(auto &v : adj[u]){
        in_degree[v]--;
        if(in_degree[v] <= 0) pq.push(v);
    }
}

```

Kosaraju Algo $O(V+E)$

Find SCC in directed graph

```

void dfs1(int v, const vvi& adj, vector<bool>& vis, vi& order) {
    used[v] = true;
    for (auto u : adj[v])
        if (!used[u])
            dfs1(u);
    order.push_back(v);
}

void dfs2(int v, const vvi& adj_inv, vector<bool>& vis, vi& component) {
    used[v] = true;
    component.push_back(v);
    for (auto u : adj_inv[v])
        if (!used[u])
            dfs2(u);
}

int main() {

```

```

int n; cin >> n;
vvi adj(n), adj_inv(n);
vi order, component;
vector<bool> vis(n, false);

for(int i = 0; i < n; i++){
    int a, b; cin >> a >> b;
    adj[a].push_back(b);
    adj_inv[b].push_back(a);
}

for (int i = 0; i < n; i++)
    if (!vis[i])
        dfs1(i);
vis.assign(n, false);

reverse(order.begin(), order.end());
for (auto u : order)
    if (!vis[u]) {
        dfs2 (u);

        // HERE! component contains all vertices in SCC

        component.clear();
    }
}

```

Edmond-Karp $O(V \cdot E^2)$

Para tener un min cut, lanzas un dfs desde s que puede ir a cualquier otro nodo siempre y cuando la arista tenga capacidad residual positiva. Esta partición te da S.

Para bipartite matching, sacas un nodo s que conectas a los blancos, y los negros los mandas a un nodo t y metes flujos.

Residual capacity := $res_cap[u][v] = \text{abs}(\text{cap}[u][v] - \text{flow}[u][v])$

```

int bfs(int s, int t, vi& parent, vvi& capacity, vvi& adj) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int> > q;
    q.push(make_pair(s, INF));

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push(make_pair(next, new_flow));
            }
        }
    }
    return 0;
}

int maxflow(int s, int t, vvi& capacity, vvi& adj) {

```

```

int flow = 0;
int n = adj.size();
vector<int> parent(n);
int new_flow;

while (new_flow = bfs(s, t, parent, capacity, adj)) {
    flow += new_flow;
    int cur = t;
    while (cur != s) {
        int prev = parent[cur];
        capacity[prev][cur] -= new_flow;
        capacity[cur][prev] += new_flow;
        cur = prev;
    }
}
return flow;
}

```

Dinic Algo $O(E \cdot V^2)$

```

struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap - edges[id].flow < 1)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
    }
}

```

```

    return level[t] != -1;
}

long long dfs(int v, long long pushed) {
    if (pushed == 0)
        return 0;
    if (v == t)
        return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u] || edges[id].cap - edges[id].flow < 1)
            continue;
        long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}
};

```

Tree Diameter

Haces un dfs desde un nodo random y calculas que nodo tiene mayor profundidad. Desde ese nodo lanzas otro dfs y la mayor profundidad es el diametro.

DYNAMIC PROGRAMMING

Kadane $O(n)$, $O(n^3)$

For circular array, compute localMin, globalMin and sum (sum of all elements in array) and then return

```
int kadane(vector<int>& vec){
    int globalMax = vec[0], localMax = vec[0];
    int globalMin = vec[0], localMin = vec[0];
    int sum = vec[0];
    for(int i = 1; i < vec.size(); i++){
        localMax = max(vec[i], localMax+vec[i]);
        globalMax = max(globalMax, localMax);

        localMin = min(vec[i], localMin+vec[i]);
        globalMin = min(globalMin, localMin);

        sum += vec[i];
    }
    /*
    In case of circular array
    if(globalMin == sum)
        return globalMax;
    else
        return max(globalMax, sum-globalMin);
    */
    return globalMax;
}

int kadane2d(vector<vector<int> >& matrix){
    int maxValue = -INF;
    for(int i = 0; i < matrix[0].size(); i++){
        vector<int> aux(matrix.size(),0);
        for(int j = i; j < matrix[0].size(); j++){
            for(int k = 0; k < aux.size(); k++) aux[k] += matrix[k][j];
            maxValue = max(maxValue, kadane(aux));
        }
    }
    return maxValue;
}
```

Longest Increasing Subsequence $O(n \log n)$

```
int lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n+1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l-1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
```

NUMBER THEORY

Sieve of Eratosthenes $O(n \log \log n)$

```
int N;
vector<bool> prime(N+1, true);
prime[0] = prime[1] = false;
for(int i = 2; i*i <= N; i++){
    if(prime[i]){
        for(int j = i*i; j <= N; j+=i)
            prime[j] = false;
    }
}
```

GCD/LCM $O(\log n)$

Id Bezout $a*x + b*y = \gcd(a,b)$

$a*b = \gcd(a,b)*\text{lcm}(a,b)$

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
ll lcm(ll a, ll b){return (a*b)/gcd(a,b);}
```

Binary Exponentiation $O(\log n)$

```
//Efficient way to calculate powers
ll binExp(ll a, ll b){
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Inverse Modular Arithmetic (pequeño teorema de fermat)

```
ll inv(ll i) {
    return i <= 1 ? i : INF - (ll)(INF/i) * inv(INF % i) % INF;
}
```

Multiplication Modular Arithmetic

If mod is around $1e9$, then just use $(a \% m) * (b \% m) \% m$ with long long to prevent overflow

```
ll moduloMultiplication(ll a, ll b, ll mod){
    ll res = 0;
    a %= mod;
    while (b) {
        if (b & 1)
            res = (res + a) % mod;
        a = (2 * a) % mod;
        b >>= 1; // b = b / 2
    }
}
```

```

    return res;
}

```

Lucas Theorem

Calculates nCk if mod := prime number

```

ll lucas(ll n, ll k, ll mod, vector<long long>& fact){
    if(n<k) return 0;
    if(n==mod) return (lucas(n%mod, k%mod,mod,fact) * lucas(n/mod, k/mod,mod,fact)) % mod;
    return (((fact[n]*mod_inv(fact[k],mod))%mod * mod_inv(fact[n-k],mod))%mod;
}

```

Chinese Remainder Theorem

$x \equiv a_1 \pmod{m_1}$

...

$x \equiv a_n \pmod{m_n}$

returns x

congruences := $\{[a_i, m_i], \dots\}$ where all m_i are pairwise coprime

```

long long chinese_remainder_theorem(vector<pair<ll,ll> > const& congruences) {
    long long M = 1;
    for (auto const& congruence : congruences)
        M *= congruence.second;

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.first;
        long long M_i = M / congruence.second;
        long long N_i = mod_inv(M_i, congruence.second);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}

```

Binomial Coefficients

$nCk = n!/(k!(n-k)!)$

$nCk = n-1Ck-1 + n-1Ck$ (1 if $k == 0$ || $k == n$)

Linear Diophantine Equation

equation with the form $a_1x_1 + \dots + a_nx_n = b$ where all a_i, x_i and b are integers admits a solution iff $(\text{GCD}(a_1, \dots, a_n) \mid b)$

GEOMETRY

Area of polygon given coordinates (coordinates must be given clockwise or counterclockwise)

```
double area(vector<pair<double, double> > coords) {
    double sum = 0;
    int n = coords.size();
    for (int i = 0; i < n; i++) {
        sum += coords[i].first * coords[(i+1)%n].second - coords[i].second * coords[(i +
1) % n].first;
    }
    return abs(sum) / 2;
}
```

```
using namespace std;
const double EPS=1e-9;
//Representar un punto
struct point {
    double x, y;
    point(){x=y=0.0;}
    point(double _x, double _y):x(_x), y(_y){}
    bool operator < (point other) const {
        if (fabs(x-other.x)>EPS)
            return x<other.x;
        return y<other.y;
    }
    bool operator==(const point &other){
        return (fabs(x-other.x)<EPS)&&(fabs(y-other.y)<EPS);
    }
};

double dist (const point &p1, const point &p2){
    return hypot(p1.x-p2.x, p1.y-p2.y);
}

double DEG_to_RAD(double d){return d*M_PI / 180.0;}
double RAD_to_DEG(double r){return r*180.0 / M_PI;}
```

```
point rotate(const point &p, double theta){
    double rad=DEG_to_RAD(theta);
    return point( p.x*cos(rad) - p.y*sin(rad),
        p.x*sin(rad)+ p.y*cos(rad));
}

struct line{double a, b, c;};

void pointsToLine(const point &p1, const point &p2, line &l){
    if (fabs(p1.x-p2.x)<EPS)
        l={1.0, 0.0, -p1.x};
    else{
        l.a=-(double)(p1.y-p2.y)/(p1.x-p2.x);
        l.b= 1.0;
        l.c= -(double)(l.a*p1.x)-p1.y;
    }
}

void pointSlopeToLine(point p, double m, line &l){
    l.a=-m;
```



```

    l.b=1.0;
    l.c=-((l.a*p.x)+(l.b*p.y));
}

bool areParalel(line l1, line l2){
    return (fabs(l1.a-l2.a)<EPS) && (fabs(l1.b-l2.b)<EPS);
}

bool areSame(line l1, line l2){
    return areParalel(l1, l2)&& fabs(l1.c-l2.c)<EPS;
}

bool areIntersect(line l1, line l2, point &p ){
    if (areParalel(l1, l2)){
        return false;
    }

    p.x=(l2.b*l1.c-l1.b*l2.c) / (l2.a*l1.b-l1.a*l2.b);
    if (fabs(l1.b)>EPS) p.y= -(l1.a*p.x +l1.c);

```

```

else
    p.y= -(l2.a*p.x + l2.c);
    return true;
}

struct vec {
    double x, y;
    vec(double _x, double _y):x(_x), y(_y){}
};

vec toVec(const point &a, const point &b){
    return vec(b.x-a.x, b.y-a.y);
}

vec scale(const vec &v, double s){
    return vec(v.x*s, v.y*s);
}

point translate(const point &p, const vec &v){
    return point(p.x+v.x, p.y+v.y);
}

double dot(const vec &a, const vec &b){
    return a.x*b.x+a.y*b.y;
}

double norm_sq(const vec &a){
    return (a.x*a.x+a.y*a.y);
}

double angle(const point &a, const point &o, const point &b){
    vec oa=toVec(o, a), ob=toVec(o, b);
    return acos(dot(oa, ob)/ sqrt(norm_sq(oa)*norm_sq(ob)));
}

double cross(vec a, vec b){
    return a.x*b.y-a.y*b.x;
}
// contra las agujas del reloj
bool ccw(point p, point q, point r){
    return cross(toVec(p, q), toVec(p, r))>EPS;
}

bool collinear(point p, point q, point r){
    return fabs(cross(toVec(p, q), toVec(p, r)))<EPS;
}

```

```

}

double distToLine(point p, point a, point b, point &c){
    vec ap=toVec(a, p), ab=toVec(a, b);
    double u =dot(ap, ab) /norm_sq(ab);
    c=translate(a, scale(ab, u));
    return dist(p, c);
}

double distToLineSegment(point p, point a, point b, point &c){
    vec ap=toVec(a, p), ab=toVec(a, b);
    double u=dot(ap, ab)/norm_sq(ab);
    if (u<0.0){
        c=point(a.x, a.y);
        return dist(p, a);
    }
    if (u>1.0){
        c=point(b.x, b.y);
        return dist(p, b);
    }
    return distToLine(p, a, b, c);
}

int insideCircle(const point &p, const point &c, int r){
    double dx=p.x-c.x, dy=p.y-c.y;
    double Euc= dx*dx+dy*dy, rSq=r*r;
    return Euc<rSq? 1 : (fabs(Euc-rSq)<EPS?0:-1);
}

```

Convex Hull $O(n \log n)$

```

struct pt {
    double x, y;
    bool operator == (pt const& t) const {
        return x == t.x && y == t.y;
    }
};

int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}

bool collinear(pt a, pt b, pt c) { return orientation(a, b, c) == 0; }

void convex_hull(vector<pt>& a, bool include_collinear = false) {
    pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.y, a.x) < make_pair(b.y, b.x);
    });
    sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
        int o = orientation(p0, a, b);
        if (o == 0)
            return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
                < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
    });
}

```

```

        return o < 0;
    });
    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 && collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !cw(st[st.size()-2], st.back(), a[i], include_collinear))
            st.pop_back();
        st.push_back(a[i]);
    }

    if (include_collinear == false && st.size() == 2 && st[0] == st[1])
        st.pop_back();

    a = st;
}

```

ADA LANGUAGE

Hello World (I/O)

```
with Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Greet is
  N : Integer;
begin
  Get (N);
  Put (N);
  -- Print "Hello, World!" to the screen
  Ada.Text_IO.Put_Line ("Hello, World!");
end Greet;
```

If/Then/Else

```
with Ada.Text_IO;          use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Check_Direction is
  N : Integer;
begin
  Put ("Enter an integer value: ");
  Get (N);
  Put (N);

  if N = 0 or N = 360 then
    Put_Line (" is due north");
  elsif N in 1 .. 89 then
    Put_Line (" is in the northeast quadrant");
  elsif N = 90 then
    Put_Line (" is due east");
  else
    Put_Line (" is not in the range 0..360");
  end if;
end Check_Direction;
```

For loop

```
with Ada.Text_IO; use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Greet_5a_Reverse is
begin
  for I in reverse 1 .. 5 loop
    Put_Line ("Hello, World!"
              & Integer'Image (I));
  end loop;
end Greet_5a_Reverse;
```

While loop

```
with Ada.Text_IO; use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Greet_5c is
  I : Integer := 1;
begin
  -- Condition must be a Boolean value
  -- (no Integers).
  -- Operator "<=" returns a Boolean
  while I <= 5 loop
    Put_Line ("Hello, World!"
              & Integer'Image (I)); -- display integer using image

    I := I + 1;
  end loop;
end Greet_5c;
```

Simple even/odd checker

```
with Ada.Text_IO;          use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Even_Odd_Check is
  -- Declare a variable to hold the user input
  Input_Number, T : Integer;
begin
  -- Prompt the user for an integer
  Get(T);
  for I in reverse 1..T loop
    Get(Input_Number);
    -- Check if the number is even or odd
    if Input_Number mod 2 = 0 then
      Put_Line("EVEN");
    else
      Put_Line("ODD");
    end if;
  end loop;
end Even_Odd_Check;
```

Compiling Ada code

Create hello.adb file with your code

```
gcc -c hello.adb
gnatbind hello
gnatlink hello
hello
```

or

```
gnatmake hello.adb
hello
```